MTS Training Resources, April 2018

Extending OpenFresco: Experimental Class APIs

Andreas Schellenberg, Ph.D., P.E.
Senior Structural Engineer, Maffei Structural Engineering







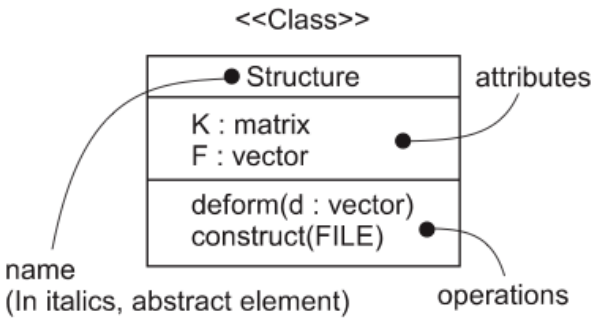
Outline of Presentation

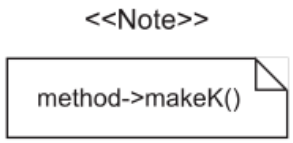
1. What is the API
2. UML
3. Class Diagram
4. Sequence Diagram
5. Methods in Experimental Classes

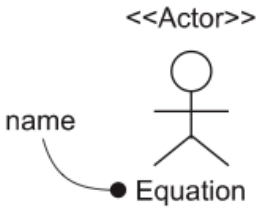
Why is the API important?

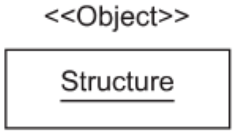
- What do each of the OpenFresco classes do and what are the requirements for changing them so they all still work together?
- Our lab uses specialized control equipment, and it may not work with OpenFresco. One of my investigators wants to implement a new form of hybrid testing. Is there something I can do to make these specialized connections?

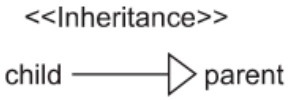
Unified Modeling Language (UML)

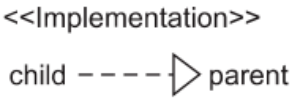


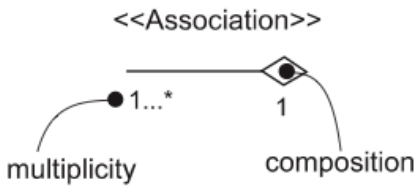


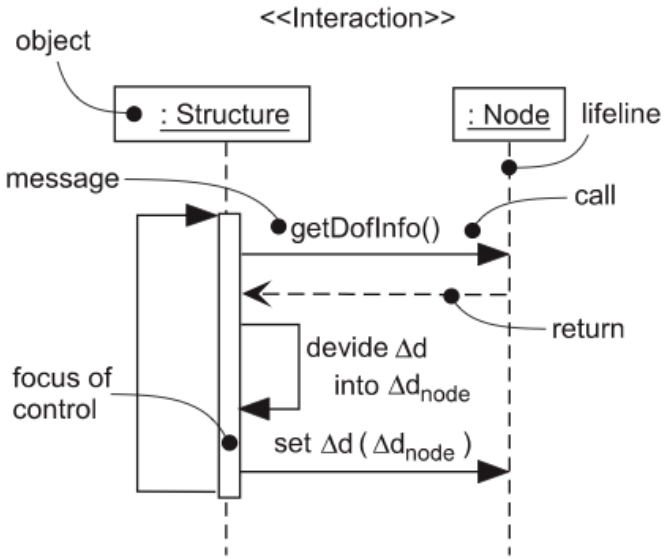




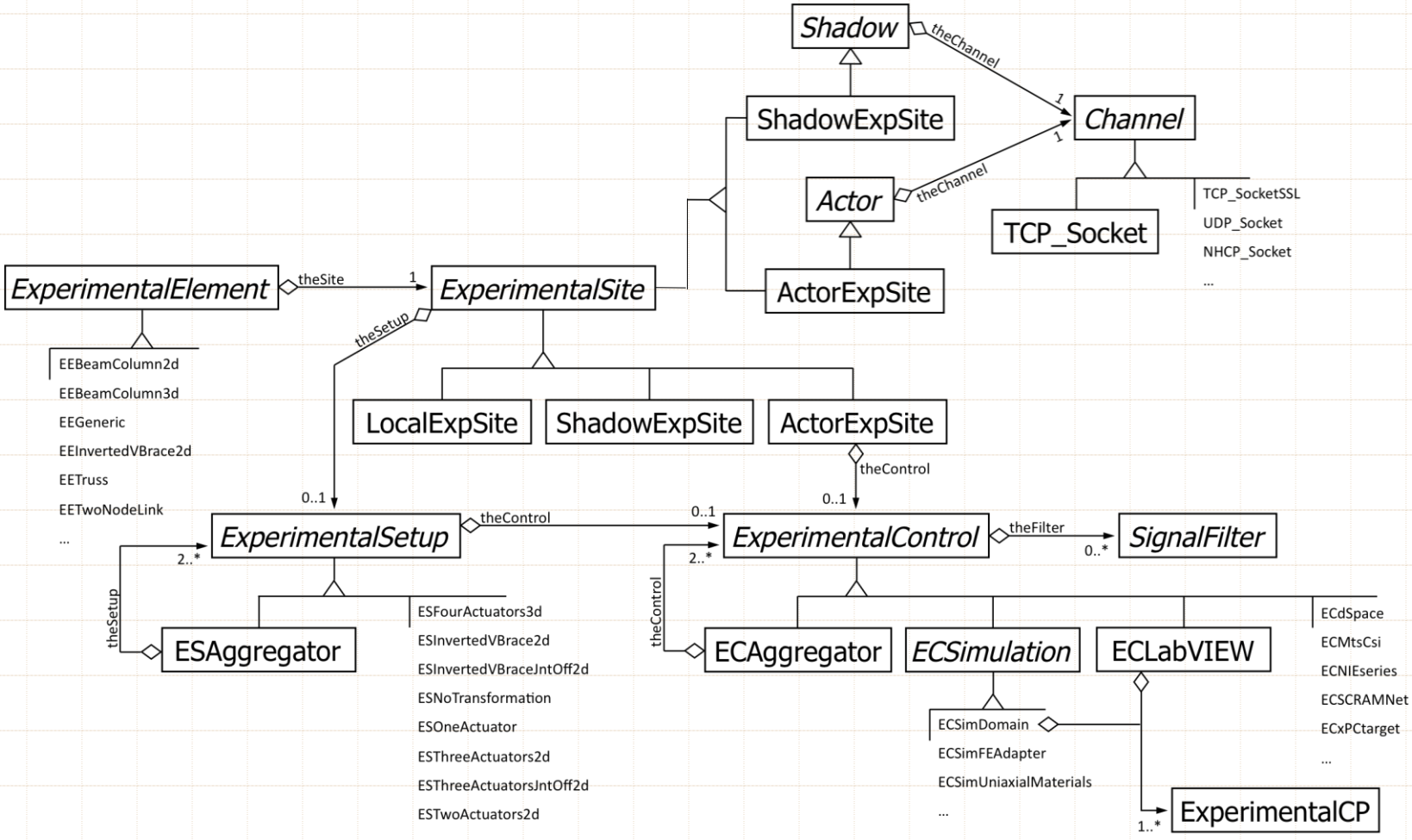




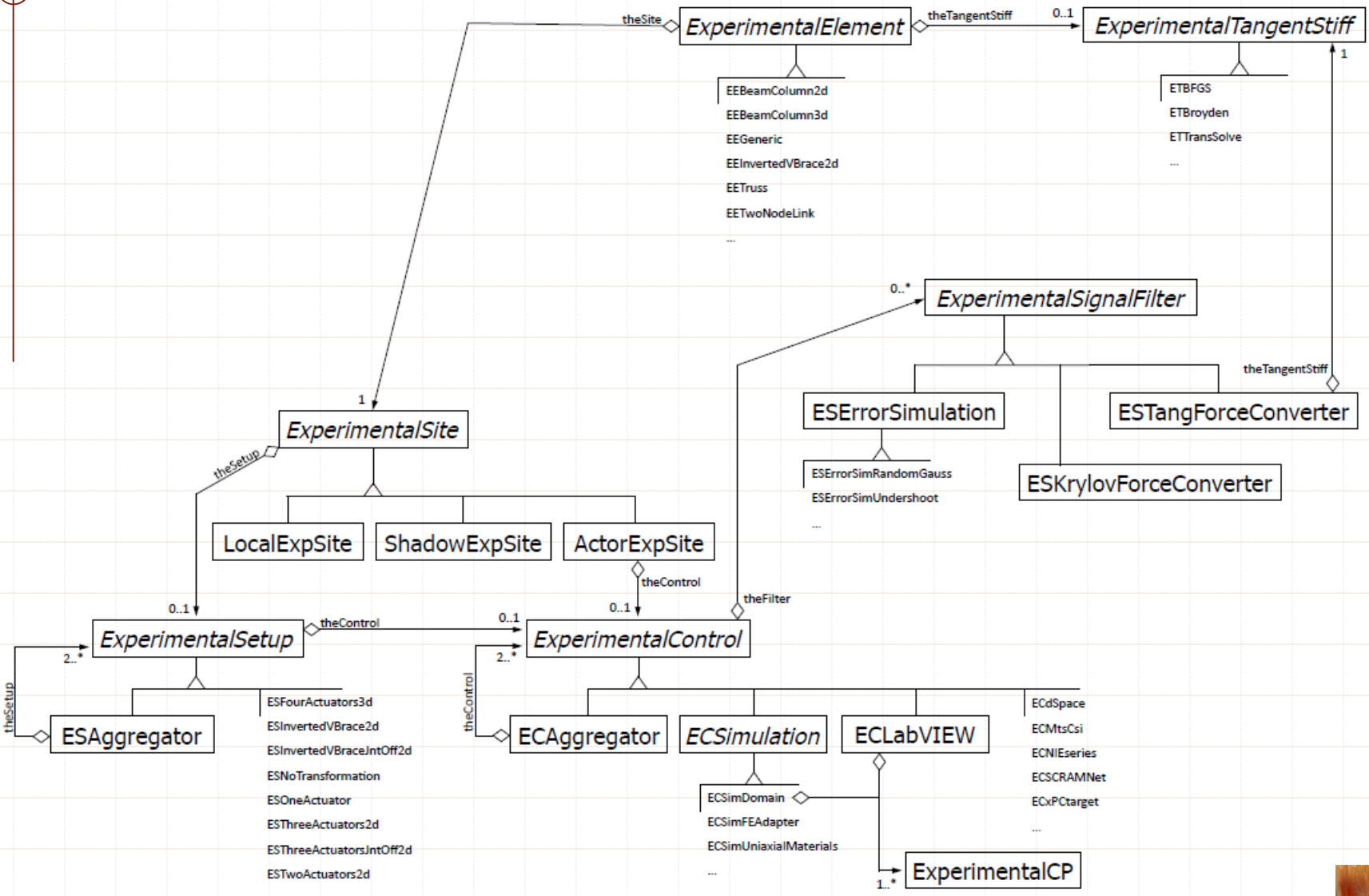




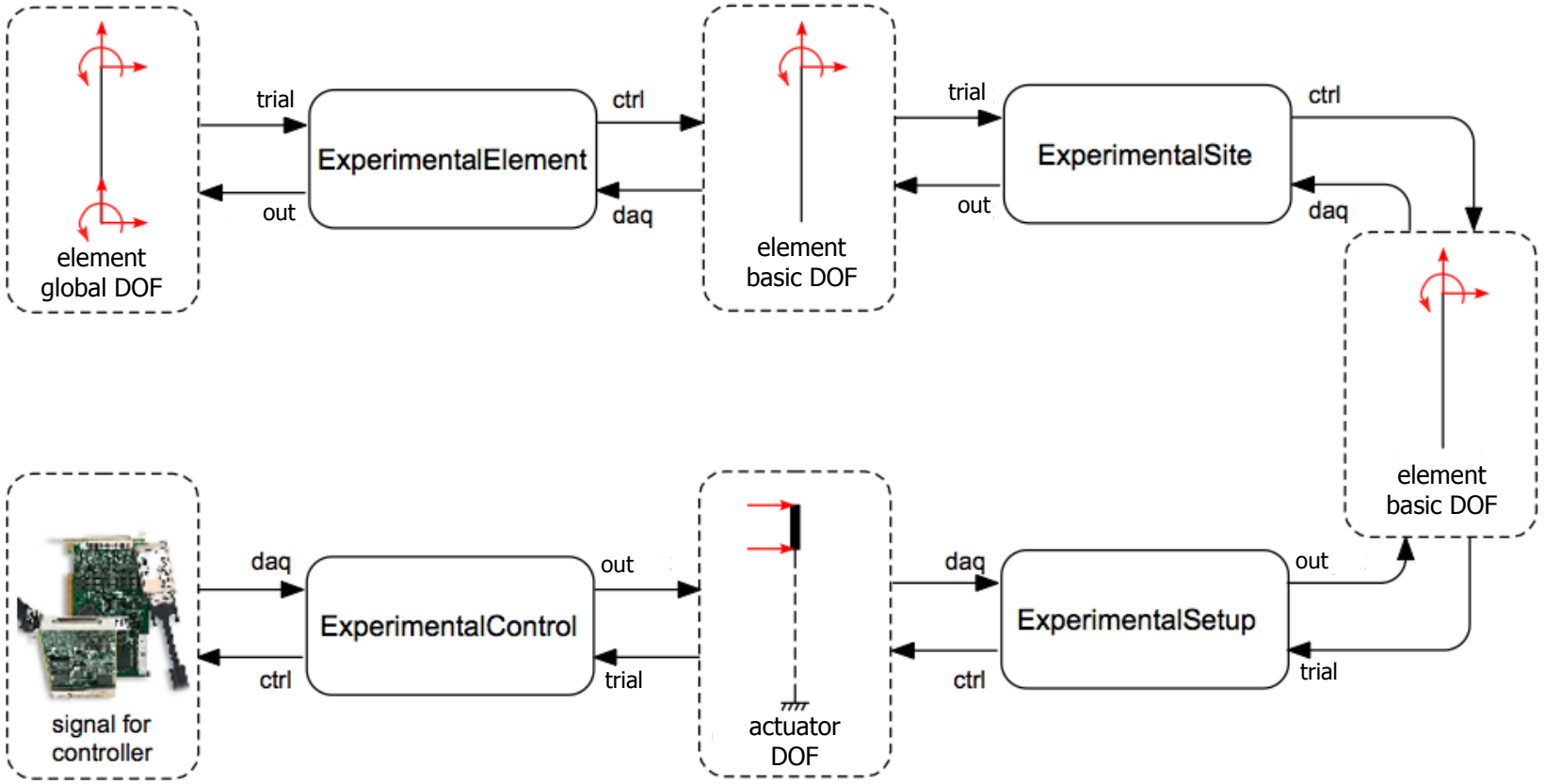
OpenFresco Class Diagram



Force and Mixed Control

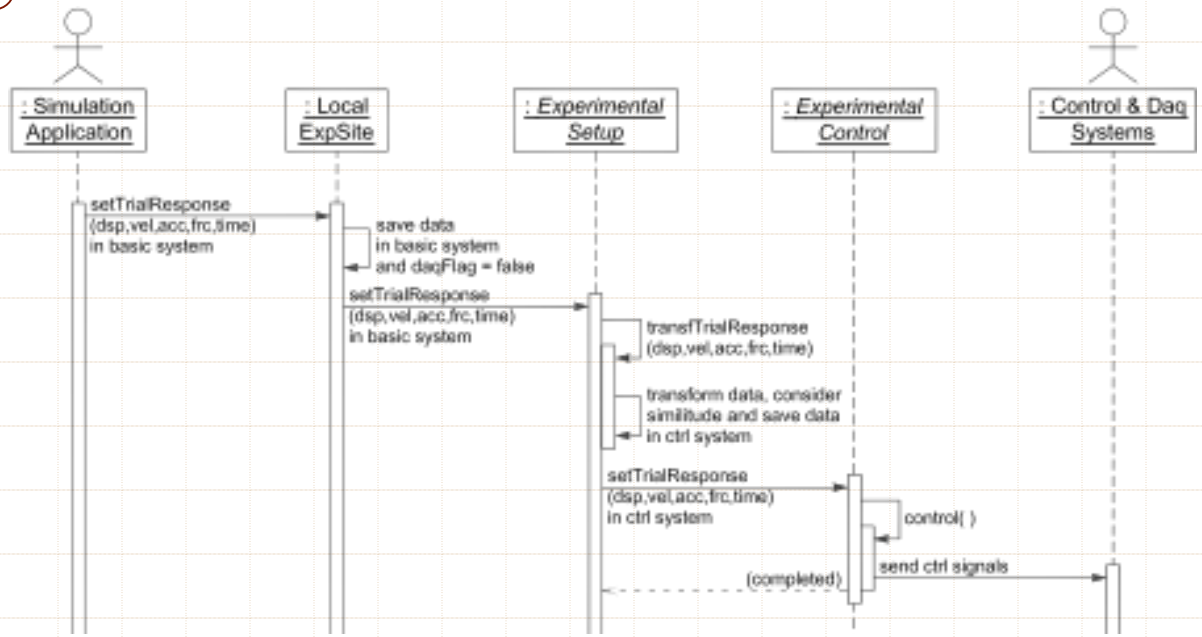


Data Transformations



Sequence Diagram for Local Test

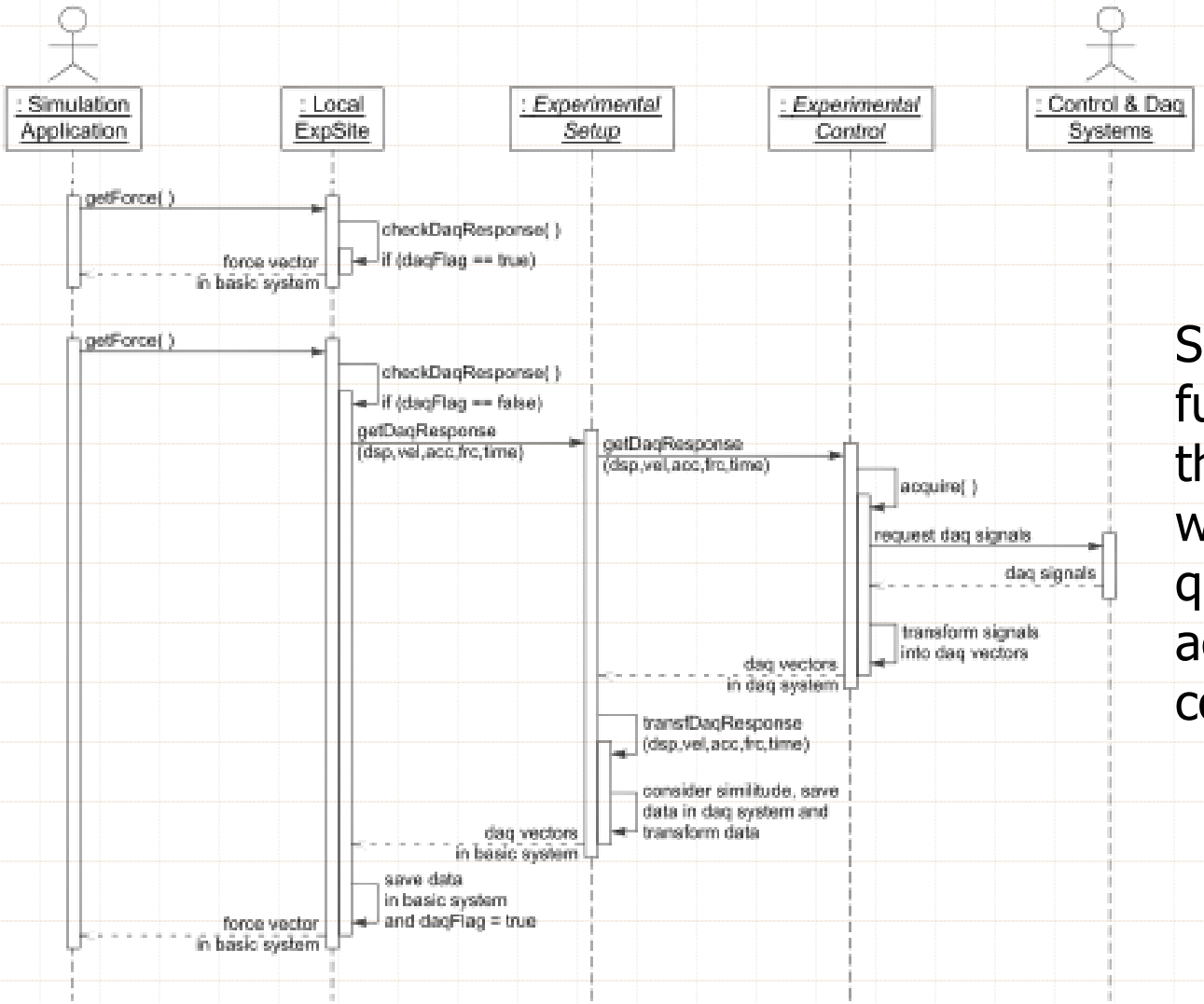
setTrialResponse()



Sequence of functions/methods that are called when trial response quantities are sent from the simulation application to the control system

Sequence Diagram for Local Test

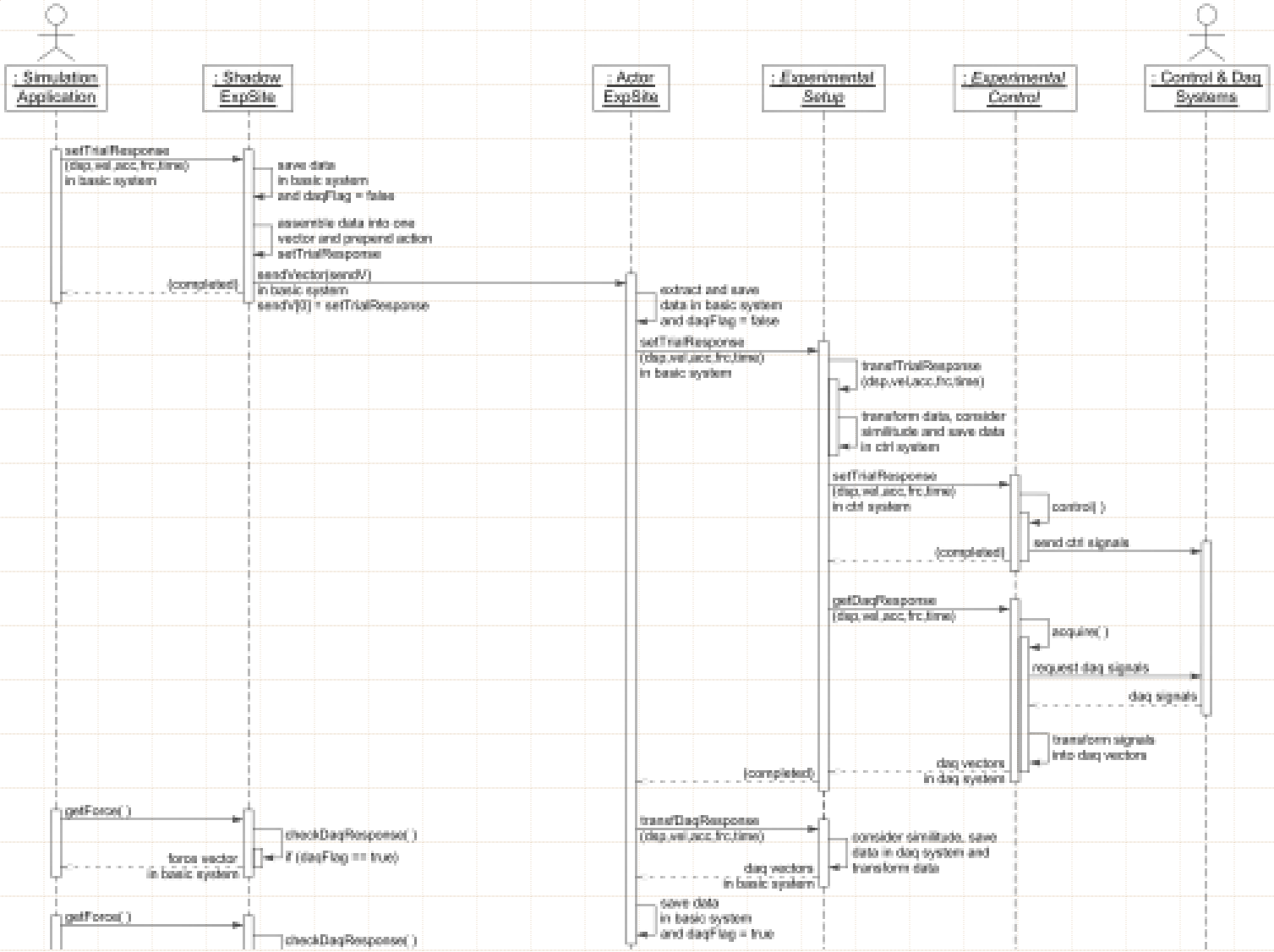
getForce()



Sequence of functions/methods that are called when response quantities are acquired from the control system

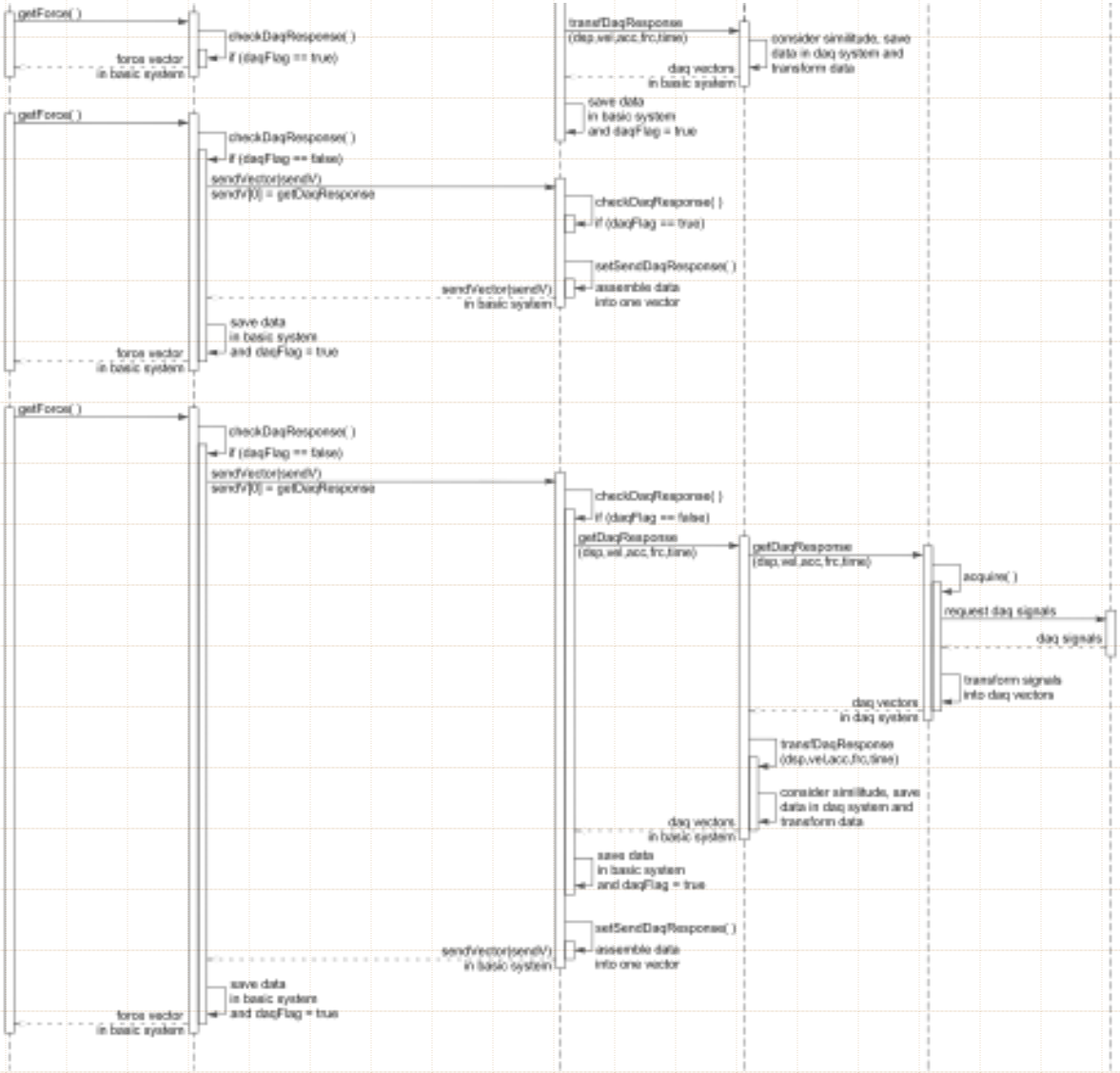
Sequence Diagram for Distributed Test

setTrialResponse() -- Experimental Setup on Server Side



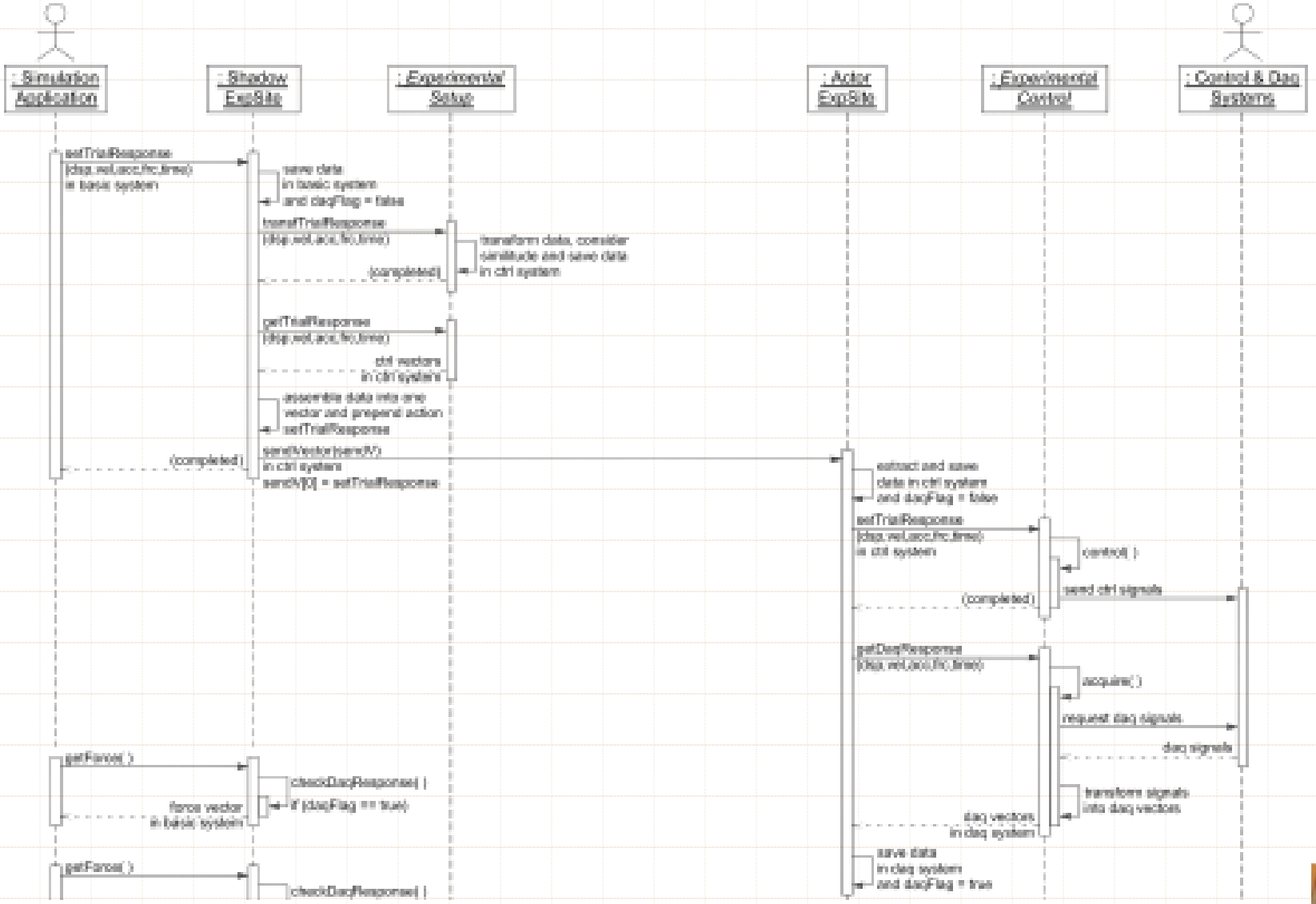
Sequence Diagram for Distributed Test

getForce() -- Experimental Setup on Server Side



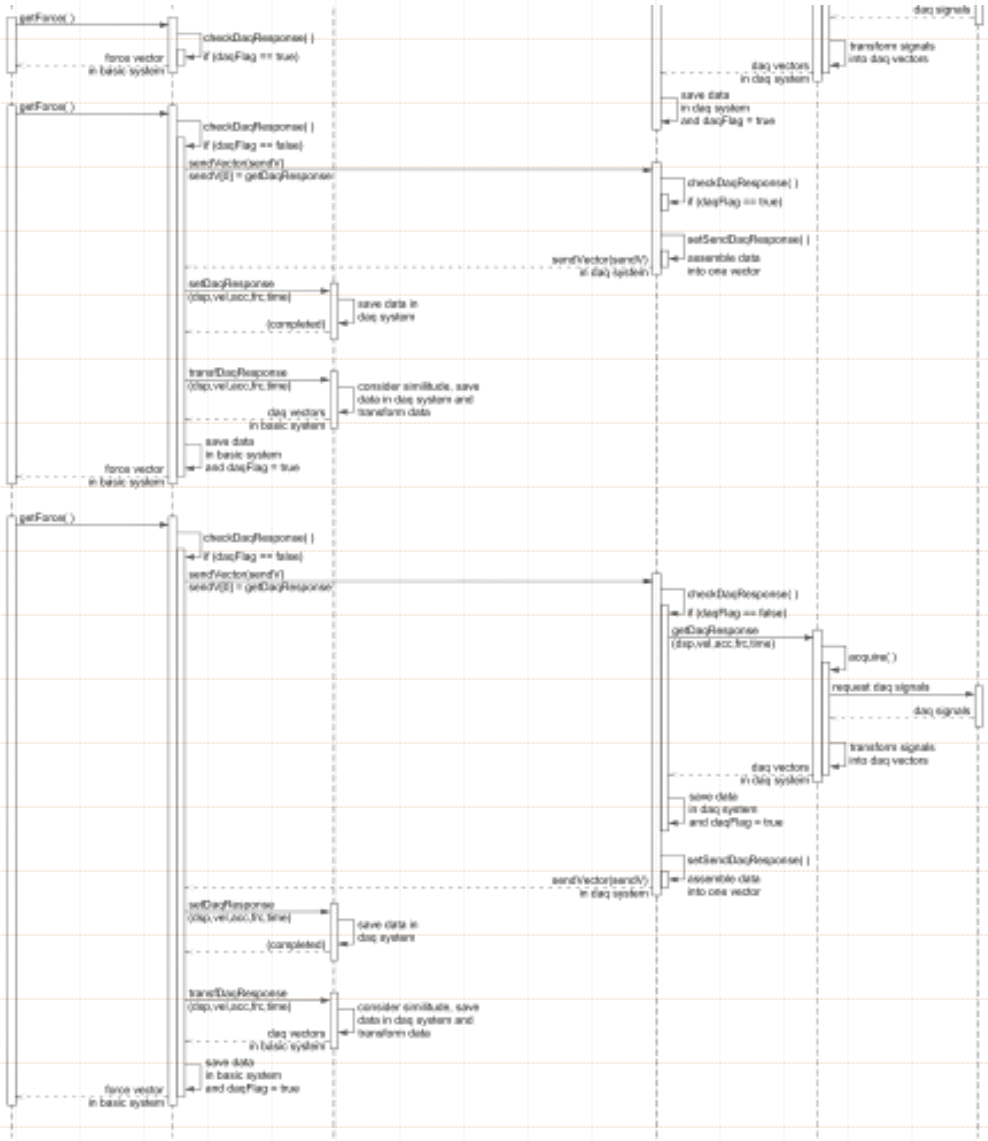
Sequence Diagram for Distributed Test

setTrialResponse() -- Experimental Setup on Client Side



Sequence Diagram for Distributed Test

getForce() -- Experimental Setup on Client Side



Experimental Element Methods

Public Member Functions

Constructor and Destructor

```
ExperimentalElement(int tag, int classTag, ExperimentalSite *site);
virtual ~ExperimentalElement( );
```

Methods dealing with nodes and number of external dof

```
virtual int getNumExternalNodes(void) const = 0;
virtual const ID &getExternalNodes(void) = 0;
virtual Node **getNodePtrs(void) = 0;
virtual int getNumDOF(void) = 0;
```

Method to obtain basic dof size; equal to the max num dof that can be controlled

```
virtual int getNumBasicDOF(void) = 0;
```

Method to set the Domain which holds the elements

```
virtual void setDomain(Domain *theDomain);
```

Methods dealing with committing and updating the solution state

```
virtual int commitState(void);
virtual int update(void);
```

Methods to set and to obtain the initial stiffness matrix

```
virtual int setInitialStiff(const Matrix &stiff) = 0;
const Matrix &getInitialStiff(void);
```

Methods to return the damping and mass matrices

```
virtual const Matrix &getDamp(void);
virtual const Matrix &getMass(void);
```

Methods for applying loads

```
virtual void zeroLoad(void) = 0;
virtual int addLoad(ElementalLoad *theLoad, double loadFactor) = 0;
virtual int addInertiaLoadToUnbalance(const Vector &accel) = 0;
virtual int setRayleighDampingFactors(double alphaM, double betaK, double betaK0, double
betaKc);
```

Methods for obtaining resisting force (force includes elemental loads)

```
virtual const Vector &getResistingForce(void) = 0;
virtual const Vector &getResistingForceIncInertia(void);
```

Methods for obtaining information specific to an element

```
virtual Response *setResponse(const char **argv, int argc, Information &eleInformation);
virtual int getResponse(int responseID, Information &eleInformation);
```

EEBeamColumn2d (constructor)

```
EEBeamColumn2d::EEBeamColumn2d(int tag, int Nd1, int Nd2,
    CrdTransf2d &coordTransf,
    ExperimentalSite *site,
    bool iM, double r)
    : ExperimentalElement(tag, ELE_TAG_EEBeamColumn2d, site),
    connectedExternalNodes(2), theCoordTransf(0),
    iMod(iM), rho(r), L(0.0),
    db(0), vb(0), ab(0), t(0),
    dbDaq(0), vbDaq(0), abDaq(0), qDaq(0), tDaq(0),
    dbCtrl(3), vbCtrl(3), abCtrl(3),
    dbPast(3), kbInit(3,3), tPast(0.0),
    firstWarning(true)
{
    // ensure the connectedExternalNode ID is of correct size & set val
    if (connectedExternalNodes.Size() != 2)  {
        opserr << "EEBeamColumn2d::EEBeamColumn2d() - element: "
            <<  tag << " failed to create an ID of size 2\n";
        exit(-1);
    }

    connectedExternalNodes(0) = Nd1;
    connectedExternalNodes(1) = Nd2;

    // set node pointers to NULL
    int i;
    for (i=0; i<2; i++)
        theNodes[i] = 0;

    // set the data size for the experimental site
    sizeCtrl = new ID(OF_Resp_All);
    sizeDaq = new ID(OF_Resp_All);

    (*sizeCtrl)[OF_Resp_Disp]  = 3;
    (*sizeCtrl)[OF_Resp_Vel]   = 3;
    (*sizeCtrl)[OF_Resp_Accel] = 3;
    (*sizeCtrl)[OF_Resp_Time]  = 1;

    (*sizeDaq)[OF_Resp_Disp]   = 3;
    (*sizeDaq)[OF_Resp_Vel]    = 3;
    (*sizeDaq)[OF_Resp_Accel]  = 3;
    (*sizeDaq)[OF_Resp_Force]  = 3;
    (*sizeDaq)[OF_Resp_Time]   = 1;

    theSite->setSize(*sizeCtrl, *sizeDaq);

    // allocate memory for trial response vectors
    db = new Vector(3);
    vb = new Vector(3);
    ab = new Vector(3);
    t  = new Vector(1);

    // allocate memory for daq response vectors
    dbDaq = new Vector(3);
    vbDaq = new Vector(3);
    abDaq = new Vector(3);
    qDaq  = new Vector(3);
    tDaq  = new Vector(1);

    // set the initial stiffness matrix size
    theInitStiff.resize(6,6);

    // get a copy of the coordinate transformation
    theCoordTransf = coordTransf.getCopy();
    if (!theCoordTransf)  {
        opserr << "EEBeamColumn2d::EEBeamColumn2d() - "
            << "failed to get copy of the coordinate transformation\n";
        exit(-1);
    }

    // initialize additional vectors
    dbCtrl.Zero();
    vbCtrl.Zero();
    abCtrl.Zero();
    dbPast.Zero();
    for (i=0; i<3; i++)  {
        qA0[i] = 0.0;
        pA0[i] = 0.0;
    }
}
```

EEBeamColumn2d::setInitialStiff

```
int EEBeamColumn2d::setInitialStiff(const Matrix &kbinit)
{
    if (kbinit.noRows() != 3 || kbinit.noCols() != 3)  {
        opserr << "EEBeamColumn2d::setInitialStiff() - "
            << "matrix size is incorrect for element: "
            << this->getTag() << endln;
        return -1;
    }
    kbInit = kbinit;

    // transform stiffness from basic sys B to basic sys A
    static Matrix kbAInit(3,3);
    kbAInit(0,0) = kbInit(0,0);
    kbAInit(1,1) = L*L*kbInit(1,1) + L*(kbInit(1,2)+kbInit(2,1)) + kbInit(2,2);
    kbAInit(1,2) = -L*kbInit(1,2) - kbInit(2,2);
    kbAInit(2,1) = -L*kbInit(2,1) - kbInit(2,2);
    kbAInit(2,2) = kbInit(2,2);

    // transform stiffness from the basic to the global system
    theInitStiff.Zero();
    theInitStiff = theCoordTransf->getInitialGlobalStiffMatrix(kbAInit);

    return 0;
}
```

EEBeamColumn2d::update

```
int EEBeamColumn2d::update()
{
    int rValue = 0;

    // get current time
    Domain *theDomain = this->getDomain();
    (*t)(0) = theDomain->getCurrentTime();

    // update the coordinate transformation
    theCoordTransf->update();

    // determine dsp, vel and acc in basic system A
    const Vector &dbA = theCoordTransf->getBasicTrialDisp();
    const Vector &vbA = theCoordTransf->getBasicTrialVel();
    const Vector &abA = theCoordTransf->getBasicTrialAccel();

    // transform displacements from basic sys A to basic sys B (nonlinear)
    (*db)[0] = (L+dbA(0))*cos(dbA(1))-L;
    (*db)[1] = -(L+dbA(0))*sin(dbA(1));
    (*db)[2] = -dbA(1)+dbA(2);
    (*vb)[0] = vbA(0)*cos(dbA(1))-(L+dbA(0))*sin(dbA(1))*vbA(1);
    (*vb)[1] = -vbA(0)*sin(dbA(1))-(L+dbA(0))*cos(dbA(1))*vbA(1);
    (*vb)[2] = -vbA(1)+vbA(2);
    (*ab)[0] = abA(0)*cos(dbA(1))-2*vbA(0)*sin(dbA(1))*vbA(1)-(L+dbA(0))*cos(dbA(1))*pow(vbA(1),2)-(L+dbA(0))*sin(dbA(1))*abA(1);
    (*ab)[1] = -abA(0)*sin(dbA(1))-2*vbA(0)*cos(dbA(1))*vbA(1)+(L+dbA(0))*sin(dbA(1))*pow(vbA(1),2)-(L+dbA(0))*cos(dbA(1))*abA(1);
    (*ab)[2] = -abA(1)+abA(2);

    if ((*db) != dbPast || (*t)(0) != tPast)  {
        // save the displacements and the time
        dbPast = (*db);
        tPast = (*t)(0);
        // set the trial response at the site
        if (theSite != 0)  {
            theSite->setTrialResponse(db, vb, ab, (Vector*)0, t);
        }
        else  {
            sData[0] = OF_RemoteTest_setTrialResponse;
            rValue += theChannel->sendVector(0, 0, *sendData, 0);
        }
    }

    return rValue;
}
```

EEBeamColumn2d::getResistingForce

```
const Vector& EEBeamColumn2d::getResistingForce()
{
    theCoordTransf->update();

    // zero the residual
    theVector.Zero();

    // get daq resisting forces
    if (theSite != 0)  {
        (*qDaq) = theSite->getForce();
    }
    else  {
        sData[0] = OF_RemoteTest_getForce;
        theChannel->sendVector(0, 0, *sendData, 0);
        theChannel->recvVector(0, 0, *recvData, 0);
    }

    // get chord rotation from basic sys A to B
    double alpha = atan2((*db)[1],L+(*db)[0]);

    // apply optional initial stiffness modification
    if (iMod == true)
        this->applyIMod();

    // use elastic axial force if axial force from test is zero
    if ((*qDaq)[0] == 0.0)  {
        double qA0 = kbInit(0,0)*(sqrt(pow(L+(*db)[0],2)+pow((*db)[1],2))-L);
        (*qDaq)[0] = cos(alpha)*qA0;
        (*qDaq)[1] += sin(alpha)*qA0;
    }

    // save corresponding ctrl response for recorder
    dbCtrl = (*db);
    vbCtrl = (*vb);
    abCtrl = (*ab);

    // transform forces from basic sys B to basic sys A (nonlinear)
    static Vector qA(3);
    qA(0) = cos(alpha)*(*qDaq)[0] + sin(alpha)*(*qDaq)[1];
    qA(1) = (*db)[1]*(*qDaq)[0] - (L+(*db)[0])*(*qDaq)[1] - (*qDaq)[2];
    qA(2) = (*qDaq)[2];

    // add fixed end forces
    for (int i=0; i<3; i++)  {
        qA(i) += qA0[i];
    }

    // Vector for reactions in basic system A
    Vector pA0Vec(pA0, 3);

    // determine resisting forces in global system
    theVector = theCoordTransf->getGlobalResistingForce(qA, pA0Vec);

    // subtract external load
    theVector.addVector(1.0, theLoad, -1.0);

    return theVector;
}
```

Experimental Setup Methods

Public Member Functions

Constructors and Destructor

```
ExperimentalSetup(int tag, ExperimentalControl* control = 0);
ExperimentalSetup(const ExperimentalSetup& es);
virtual ~ExperimentalSetup();
```

Methods dealing with data sizes and setup

```
virtual int setSize(ID sizeT, ID sizeO) = 0;
virtual int setup( ) = 0;
virtual int setCtrlDaqSize( );
```

Methods dealing with setting and getting responses

```
virtual int setTrialResponse(const Vector* disp, const Vector* vel, const Vector* accel,
                             const Vector* force, const Vector* time);
virtual int getTrialResponse(Vector* disp, Vector* vel, Vector* accel,
                             Vector* force, Vector* time);
virtual int setDaqResponse(const Vector* disp, const Vector* vel, const Vector* accel,
                           const Vector* force, const Vector* time);
virtual int getDaqResponse(Vector* disp, Vector* vel, Vector* accel,
                           Vector* force, Vector* time);
```

Methods to transform the response

```
virtual int transfTrialResponse(const Vector* disp, const Vector* vel, const Vector* accel,
                                const Vector* force, const Vector* time);
virtual int transfDaqResponse(Vector* disp, Vector* vel, Vector* accel,
                              Vector* force, Vector* time);
```

Method to commit the solution state

```
virtual int commitState( );
```

Method to obtain a copy of the setup

```
virtual ExperimentalSetup *getCopy( ) = 0;
```

Methods to set the control and data acquisition factors

```
void setCtrlDispFactor(const Vector& f);
void setCtrlVelFactor(const Vector& f);
void setCtrlAccelFactor(const Vector& f);
void setCtrlForceFactor(const Vector& f);
void setCtrlTimeFactor(const Vector& f);

void setDaqDispFactor(const Vector& f);
void setDaqVelFactor(const Vector& f);
void setDaqAccelFactor(const Vector& f);
void setDaqForceFactor(const Vector& f);
void setDaqTimeFactor(const Vector& f);
```

Methods to obtain the response sizes

```
virtual ID getCtrlSize( );
virtual ID getDaqSize( );
virtual int getCtrlSize(int rType);
virtual int getDaqSize(int rType);
```

Protected Member Functions

Methods to transform the responses

```
virtual int transfTrialDisp(const Vector* disp) = 0;
virtual int transfTrialVel(const Vector* vel) = 0;
virtual int transfTrialAccel(const Vector* accel) = 0;
virtual int transfTrialForce(const Vector* force) = 0;
virtual int transfTrialTime(const Vector* time) = 0;

virtual int transfDaqDisp(const Vector* disp) = 0;
virtual int transfDaqVel(const Vector* vel) = 0;
virtual int transfDaqAccel(const Vector* accel) = 0;
virtual int transfDaqForce(const Vector* force) = 0;
virtual int transfDaqTime(const Vector* time) = 0;
```

Methods to set the response sizes

```
virtual void setCtrl( );
virtual void setDaq( );
```

ESTwoActuators2d (constructor)

```
ESTwoActuators2d::ESTwoActuators2d(int tag,
    double actLength0, double actLength1,
    double rigidLength,
    ExperimentalControl* control,
    int nlgeom, char *posact, double philocx)
    : ExperimentalSetup(tag, control),
    La0(actLength0), La1(actLength1), L(rigidLength),
    nlGeom(nlgeom), phiLocX(philocx), rotLocX(3,3)
{
    strcpy(posAct,posact);

    // call setup method
    this->setup();
}
```

```
int ESTwoActuators2d::setup()
{
    // setup for ctrl/daq vectors of ESTwoActuators2d
    sizeCtrl->Zero();
    sizeDaq->Zero();
    for (int i=0; i<OF_Resp_Time; i++) {
        (*sizeCtrl)[i] = 2;
        (*sizeDaq)[i] = 2;
    }
    (*sizeCtrl)[OF_Resp_Time] = 1;
    (*sizeDaq)[OF_Resp_Time] = 1;

    this->setCtrlDaqSize();

    // initialize rotation matrix
    rotLocX.Zero();
    double pi = acos(-1.0);
    rotLocX(0,0) = cos(phiLocX/180.0*pi); rotLocX(0,1) = -sin(phiLocX/180.0*pi);
    rotLocX(1,0) = sin(phiLocX/180.0*pi); rotLocX(1,1) =  cos(phiLocX/180.0*pi);
    rotLocX(2,2) = 1.0;

    return OF_ReturnType_completed;
}
```

ESTwoActuators2d::setSize

```
int ESTwoActuators2d::setSize(ID sizeT, ID sizeO)
{
    // check sizeTrial and sizeOut
    // for ESTwoActuators2d object

    // a component of sizeT/sizeO must be equal
    // to 3 if it is non-zero.
    for (int i=0; i<OF_Resp_Time; i++) {
        if ((sizeT[i] != 0 && sizeT[i] != 3) ||
            (sizeO[i] != 0 && sizeO[i] != 3)) {
            opserr << "ESTwoActuators2d::setSize - wrong sizeTrial/Out\n";
            opserr << "see User Manual.\n";
            opserr << "sizeT = " << sizeT;
            opserr << "sizeO = " << sizeO;
            return OF_ReturnType_failed;
        }
    }
    if ((sizeT[OF_Resp_Time] != 0 && sizeT[OF_Resp_Time] != 1) ||
        (sizeO[OF_Resp_Time] != 0 && sizeO[OF_Resp_Time] != 1)) {
        opserr << "ESTwoActuators2d::setSize - wrong sizeTrial/Out\n";
        opserr << "see User Manual.\n";
        opserr << "sizeT = " << sizeT;
        opserr << "sizeO = " << sizeO;
        return OF_ReturnType_failed;
    }

    return OF_ReturnType_completed;
}
```

ESTwoActuators2d::transfTrialResponse

```
int ESTwoActuators2d::transfTrialResponse(const Vector* disp,
    const Vector* vel,
    const Vector* accel,
    const Vector* force,
    const Vector* time)
{
    // transform data
    if (disp != 0) {
        this->transfTrialDisp(disp);
        for (int i=0; i<(*sizeCtrl)[OF_Resp_Disp]; i++)
            (*cDisp)[i] *= (*cDispFact)[i];
    }
    if (disp != 0 && vel != 0) {
        this->transfTrialVel(disp,vel);
        for (int i=0; i<(*sizeCtrl)[OF_Resp_Vel]; i++)
            (*cVel)[i] *= (*cVelFact)[i];
    }
    if (disp != 0 && vel != 0 && accel != 0) {
        this->transfTrialAccel(disp,vel,accel);
        for (int i=0; i<(*sizeCtrl)[OF_Resp_Accel]; i++)
            (*cAccel)[i] *= (*cAccelFact)[i];
    }
    if (force != 0) {
        this->transfTrialForce(force);
        for (int i=0; i<(*sizeCtrl)[OF_Resp_Force]; i++)
            (*cForce)[i] *= (*cForceFact)[i];
    }
    if (time != 0) {
        this->transfTrialTime(time);
        for (int i=0; i<(*sizeCtrl)[OF_Resp_Time]; i++)
            (*cTime)[i] *= (*cTimeFact)[i];
    }

    return OF_ReturnType_completed;
}
```

```
int ESTwoActuators2d::transfTrialDisp(const Vector* disp)
{
    // rotate direction
    static Vector d(3);
    d = rotLocX*(*disp);

    // linear geometry, actuators left
    if (nlGeom == 0 && strcmp(posAct,"left") == 0)  {
        // actuator 0
        (*cDisp)(0) = d(0);
        // actuator 1
        (*cDisp)(1) = d(0) - L*d(2);
    }
    // linear geometry, actuators right
    else if (nlGeom == 0 && strcmp(posAct,"right") == 0)  {
        // actuator 0
        (*cDisp)(0) = -d(0);
        // actuator 1
        (*cDisp)(1) = -d(0) + L*d(2);
    }
    // nonlinear geometry, actuators left
    else if (nlGeom == 1 && strcmp(posAct,"left") == 0)  {
        // actuator 0
        (*cDisp)(0) = d(0);
        // actuator 1
        (*cDisp)(1) = pow(pow(d(0)-L*sin(d(2))+La1,2.0)+pow(L*
    }
    // nonlinear geometry, actuators right
    else if (nlGeom == 1 && strcmp(posAct,"right") == 0)  {
        // actuator 0
        (*cDisp)(0) = -d(0);
        // actuator 1
        (*cDisp)(1) = pow(pow(-d(0)+L*sin(d(2))+La1,2.0)+pow(L
    }

    return OF_ReturnType_completed;
}
```

ESTwoActuators2d::transfDaqResponse

```
int ExperimentalSetup::transfDaqResponse(Vector* disp,
    Vector* vel,
    Vector* accel,
    Vector* force,
    Vector* time)
{
    // transform data
    if (disp != 0) {
        for (int i=0; i<(*sizeDaq)[OF_Resp_Disp]; i++)
            (*dDisp)[i] *= (*dDispFact)[i];
        this->transfDaqDisp(disp);
    }
    if (vel != 0) {
        for (int i=0; i<(*sizeDaq)[OF_Resp_Vel]; i++)
            (*dVel)[i]  *= (*dVelFact)[i];
        this->transfDaqVel(vel);
    }
    if (accel != 0) {
        for (int i=0; i<(*sizeDaq)[OF_Resp_Accel]; i++)
            (*dAccel)[i] *= (*dAccelFact)[i];
        this->transfDaqAccel(accel);
    }
    if (force != 0) {
        for (int i=0; i<(*sizeDaq)[OF_Resp_Force]; i++)
            (*dForce)[i] *= (*dForceFact)[i];
        this->transfDaqForce(force);
    }
    if (time != 0) {
        for (int i=0; i<(*sizeDaq)[OF_Resp_Time]; i++)
            (*dTime)[i] *= (*dTimeFact)[i];
        this->transfDaqTime(time);
    }

    return OF_ReturnType_completed;
}
```

```
int ESTwoActuators2d::transfDaqForce(Vector* force)
{
    // linear geometry, actuators left
    if (nlGeom == 0 && strcmp(posAct,"left") == 0)  {
        (*force)(0) = (*dForce)(0) + (*dForce)(1);
        (*force)(1) = 0.0;
        (*force)(2) = -L*(*dForce)(1);
    }
    // linear geometry, actuators right
    else if (nlGeom == 0 && strcmp(posAct,"right") == 0)  {
        (*force)(0) = -(*dForce)(0) - (*dForce)(1);
        (*force)(1) = 0.0;
        (*force)(2) = L*(*dForce)(1);
    }
    // nonlinear geometry, actuators left
    else if (nlGeom == 1 && strcmp(posAct,"left") == 0)  {
        double d0 = La1 + (*dDisp)(0);
        double d1 = La1 + (*dDisp)(1);

        double disp2 = atan2(d0,L) - acos((d1*d1-2*L*L-d0*d0)/(-2*L*pow(L*L
        double theta1 = asin(L*(1.0-cos(disp2))/d1);

        double fx1 = (*dForce)(1)*cos(theta1);
        double fy1 = (*dForce)(1)*sin(theta1);

        (*force)(0) = (*dForce)(0) + fx1;
        (*force)(1) = 0.0;
        (*force)(2) = -fx1*L*cos(disp2) - fy1*L*sin(disp2);
    }
    // nonlinear geometry, actuators right
    else if (nlGeom == 1 && strcmp(posAct,"right") == 0)  {
        double d0 = La1 + (*dDisp)(0);
        double d1 = La1 + (*dDisp)(1);

        double disp2 = acos((d1*d1-2*L*L-d0*d0)/(-2*L*pow(L*L+d0*d0,0.5)))
        double theta1 = asin(L*(1.0-cos(disp2))/d1);

        double fx1 = (*dForce)(1)*cos(theta1);
        double fy1 = (*dForce)(1)*sin(theta1);

        (*force)(0) = -(*dForce)(0) - fx1;
        (*force)(1) = 0.0;
        (*force)(2) = fx1*L*cos(disp2) + fy1*L*sin(disp2);
    }
```

Experimental Control Methods

Public Member Functions

Constructors and Destructor

```
ExperimentalControl(int tag);
ExperimentalControl(const ExperimentalControl& ec);
virtual ~ExperimentalControl( );
```

Methods dealing with data sizes and setup

```
virtual int setSize(ID sizeT, ID sizeO) = 0;
virtual int setup( ) = 0;
```

Methods dealing with setting and getting responses

```
virtual int setTrialResponse(const Vector* disp, const Vector* vel, const Vector* accel,
                             const Vector* force, const Vector* time);
virtual int getDaqResponse(Vector* disp, Vector* vel, Vector* accel,
                           Vector* force, Vector* time);
```

Method to commit the solution state

```
virtual int commitState( );
```

Method to obtain a copy of the control

```
virtual ExperimentalControl *getCopy( ) = 0;
```

Method to add a data filter

```
void addFilter(SignalFilter& f);
```

Protected Member Functions

Methods to communicate with the control and data acquisition systems

```
virtual int control( ) = 0;
    virtual int acquire( ) = 0;
```

ECSCRAMNet (constructor)

```
ECSCRAMNet::ECSCRAMNet(int tag, int memoffset, int numactch)
    : ExperimentalControl(tag),
    memOffset(memoffset), numActCh(numactch),
    memPtrBASE(0), memPtrOPF(0),
    newTarget(0), switchPC(0), atTarget(0),
    ctrlDisp(0), ctrlVel(0), ctrlAccel(0), ctrlForce(0), ctrlTime(0),
    daqDisp(0), daqVel(0), daqAccel(0), daqForce(0), daqTime(0)
{
#ifdef _WIN32
    // map the SCRAMNet control status registers (CSRs)
    // and the SCRAMNet physical memory
    int rValue = sp_scram_init();
    if (rValue != 0)  {
        opserr << "ECSCRAMNet::ECSCRAMNet() - sp_scram_init:";
        if (rValue == -4)
            opserr << " could not open driver.\n";
        if (rValue == -3)
            opserr << " could not map memory.\n";
        if (rValue == -2)
            opserr << " could not map CSRs.\n";
        if (rValue == -1)
            opserr << " could not read registry.\n";
        exit(rValue);
    }
#else
    // map the SCRAMNet control status registers (CSRs)
    rValue = scr_reg_mm(MAP);
    if (rValue != 0)  {
        opserr << "ECSCRAMNet::ECSCRAMNet() - scr_reg_mm(MAP):"
            << " could not map CSRs.\n";
        exit(rValue);
    }

    // map the SCRAMNet physical memory
    rValue = scr_mem_mm(MAP);
    if (rValue != 0)  {
        opserr << "ECSCRAMNet::ECSCRAMNet() - scr_mem_mm(MAP):"
            << " could not map memory.\n";
        scr_reg_mm(UNMAP);
        exit(rValue);
    }
#endif
```

```
    // set CSR0 bits so node can receive and transmit data
    unsigned short csrValue = 0x8003;
    scr_csr_write(SCR_CSR0,csrValue);

    // get the pointer to the base memory address
    memPtrBASE = (int*) get_base_mem();

    // get address for OpenFresco memory on SCRAMNet board
    // memOffset is given in bytes, so divide by 4
    memPtrOPF = (float*) (memPtrBASE + (memOffset/4));
    float *memPtr = memPtrOPF;

    // setup pointers to state flags
    newTarget = (unsigned int*) memPtr;  memPtr++;
    switchPC  = (unsigned int*) memPtr;  memPtr++;
    atTarget  = (unsigned int*) memPtr;  memPtr++;

    // setup pointers to control memory locations
    ctrlDisp  = memPtr;  memPtr += numActCh;
    ctrlVel   = memPtr;  memPtr += numActCh;
    ctrlAccel = memPtr;  memPtr += numActCh;
    ctrlForce = memPtr;  memPtr += numActCh;
    ctrlTime  = memPtr;  memPtr += numActCh;

    // setup pointers to daq memory locations
    daqDisp  = memPtr;  memPtr += numActCh;
    daqVel   = memPtr;  memPtr += numActCh;
    daqAccel = memPtr;  memPtr += numActCh;
    daqForce = memPtr;  memPtr += numActCh;
    daqTime  = memPtr;  memPtr += numActCh;

    // initialize everything to zero
    newTarget[0] = 0;
    switchPC[0]  = 0;
    atTarget[0]  = 0;
    for (int i=0; i<numActCh; i++)  {
        ctrlDisp[i]  = 0.0;  daqDisp[i]  = 0.0;
        ctrlVel[i]   = 0.0;  daqVel[i]   = 0.0;
        ctrlAccel[i] = 0.0;  daqAccel[i] = 0.0;
        ctrlForce[i] = 0.0;  daqForce[i] = 0.0;
        ctrlTime[i]  = 0.0;  daqTime[i]  = 0.0;
    }

    opserr << "************************************************\n";
    opserr << "* The SCRANNet csr and memory have been mapped *\n";
    opserr << "************************************************\n";
    opserr << endln;
}
```

ECSCRAMNet::setSize

```
int ECSCRAMNet::setSize(ID sizeT, ID sizeO)
{
    // check sizeTrial and sizeOut
    // for ECSCRAMNet object

    // ECSCRAMNet objects can only use
    // trial response vectors with size <= numActCh and
    // output response vectors with size <= numActCh
    // check these are available in sizeT/sizeO.
    if (sizeT(OF_Resp_Disp) > numActCh || sizeT(OF_Resp_Vel) > numActCh ||
        sizeT(OF_Resp_Accel) > numActCh || sizeT(OF_Resp_Force) > numActCh ||
        sizeT(OF_Resp_Time) > numActCh ||
        sizeO(OF_Resp_Disp) > numActCh || sizeO(OF_Resp_Vel) > numActCh ||
        sizeO(OF_Resp_Accel) > numActCh || sizeO(OF_Resp_Force) > numActCh ||
        sizeO(OF_Resp_Time) > numActCh)  {
        opserr << "ECSCRAMNet::setSize() - wrong sizeTrial/Out\n";
        opserr << "see User Manual.\n";
        scr_reg_mm(UNMAP);
        scr_mem_mm(UNMAP);
        exit(OF_ReturnType_failed);
    }

    (*sizeCtrl) = sizeT;
    (*sizeDaq)  = sizeO;

    return OF_ReturnType_completed;
}
```

ECSCRAMNet::setTrialResponse

```
int ECSCRAMNet::setTrialResponse(const Vector* disp,
    const Vector* vel,
    const Vector* accel,
    const Vector* force,
    const Vector* time)
{
    int i, rValue = 0;
    if (disp != 0)  {
        for (i=0; i<(*sizeCtrl)(OF_Resp_Disp); i++)  {
            ctrlDisp[i] = float((*disp)(i));
            if (theCtrlFilters[OF_Resp_Disp] != 0)
                ctrlDisp[i] = float(theCtrlFilters[OF_Resp_Disp]->filtering(ctrlDisp[i]));
        }
    }
    if (vel != 0)  {
        for (i=0; i<(*sizeCtrl)(OF_Resp_Vel); i++)  {
            ctrlVel[i] = float((*vel)(i));
            if (theCtrlFilters[OF_Resp_Vel] != 0)
                ctrlVel[i] = float(theCtrlFilters[OF_Resp_Vel]->filtering(ctrlVel[i]));
        }
    }
    if (accel != 0)  {
        for (i=0; i<(*sizeCtrl)(OF_Resp_Accel); i++)  {
            ctrlAccel[i] = float((*accel)(i));
            if (theCtrlFilters[OF_Resp_Accel] != 0)
                ctrlAccel[i] = float(theCtrlFilters[OF_Resp_Accel]->filtering(ctrlAccel[i]));
        }
    }
    if (force != 0)  {
        for (i=0; i<(*sizeCtrl)(OF_Resp_Force); i++)  {
            ctrlForce[i] = float((*force)(i));
            if (theCtrlFilters[OF_Resp_Force] != 0)
                ctrlForce[i] = float(theCtrlFilters[OF_Resp_Force]->filtering(ctrlForce[i]));
        }
    }
    if (time != 0)  {
        for (i=0; i<(*sizeCtrl)(OF_Resp_Time); i++)  {
            ctrlTime[i] = float((*time)(i));
            if (theCtrlFilters[OF_Resp_Time] != 0)
                ctrlTime[i] = float(theCtrlFilters[OF_Resp_Time]->filtering(ctrlTime[i]));
        }
    }

    rValue = this->control();

    return rValue;
}
```

```
int ECSCRAMNet::control()
{
    // set newTarget flag
    newTarget[0] = 1;

    // wait until switchPC flag has changed as well
    flag = 0;
    while (flag != 1) {
        // read switchPC flag
        flag = switchPC[0];
    }

    // reset newTarget flag
    newTarget[0] = 0;

    // wait until switchPC flag has changed as well
    flag = 1;
    while (flag != 0) {
        // read switchPC flag
        flag = switchPC[0];
    }

    return OF_ReturnType_completed;
}
```

ECSCRAMNet::getDaqResponse

```
int ECSCRAMNet::getDaqResponse(Vector* disp,
    Vector* vel,
    Vector* accel,
    Vector* force,
    Vector* time)
{
    this->acquire();

    int i;
    if (disp != 0)  {
        for (i=0; i<(*sizeDaq)(OF_Resp_Disp); i++)  {
            if (theDaqFilters[OF_Resp_Disp] != 0)
                daqDisp[i] = float(theDaqFilters[OF_Resp_Disp]->filtering(daqDisp[i]));
            (*disp)(i) = daqDisp[i];
        }
    }
    if (vel != 0)  {
        for (i=0; i<(*sizeDaq)(OF_Resp_Vel); i++)  {
            if (theDaqFilters[OF_Resp_Vel] != 0)
                daqVel[i] = float(theDaqFilters[OF_Resp_Vel]->filtering(daqVel[i]));
            (*vel)(i) = daqVel[i];
        }
    }
    if (accel != 0)  {
        for (i=0; i<(*sizeDaq)(OF_Resp_Accel); i++)  {
            if (theDaqFilters[OF_Resp_Accel] != 0)
                daqAccel[i] = float(theDaqFilters[OF_Resp_Accel]->filtering(daqAccel[i]));
            (*accel)(i) = daqAccel[i];
        }
    }
    if (force != 0)  {
        for (i=0; i<(*sizeDaq)(OF_Resp_Force); i++)  {
            if (theDaqFilters[OF_Resp_Force] != 0)
                daqForce[i] = float(theDaqFilters[OF_Resp_Force]->filtering(daqForce[i]));
            (*force)(i) = daqForce[i];
        }
    }
    if (time != 0)  {
        for (i=0; i<(*sizeDaq)(OF_Resp_Time); i++)  {
            if (theDaqFilters[OF_Resp_Time] != 0)
                daqTime[i] = float(theDaqFilters[OF_Resp_Time]->filtering(daqTime[i]));
            (*time)(i) = daqTime[i];
        }
    }

    return OF_ReturnType_completed;
}
```

```
int ECSCRAMNet::acquire()
{
    // wait until target is reached
    flag = 0;
    while (flag != 1) {
        // read atTarget flag
        flag = atTarget[0];
    }

    return OF_ReturnType_completed;
}
```

Questions?
Discussion





http://openfresco.berkeley.edu/

MAFFEI
STRUCTURAL ENGINEERING

MTS®